Atividade 09 – Ethernet e MAC

Nome: Matheus Milanezi
Disciplina: Redes de Computadores: Internetworking, Roteamento e Transmissão
Ferramentas: Wireshark 4.6.8 (Npcap) no Windows 10; Python 3.14 + Scapy 2.8.0 (scripts próprios)
Captura: `captura_atividade9.pcapng` – 21/10/2026, das 20:07:01 às 20:09:23 (141,7 s), 13 936 quadros

1. Captura e identificação dos quadros Ethernet

Interface utilizada

Item	Valor
Interface	**Ethernet** – Intel(R) Ethernet Connection (2) I219-V, 1 Gbit/s, cabo até o roteador
Identificador no Wireshark	`\Device\NPF_{5E0E9B59-734A-4CDF-AA27-8E1A6FD6B41B}`
MAC do PC	`e0:d5:5e:f4:8e:fa` (OUI `e0:d5:5e` = Gigabyte, fabricante da placa-mãe onde a placa de rede é integrada; o Wireshark mostra `GigaByteTech_f4:8e:fa`)
IP do PC	`192.168.1.2/24`
Gateway	`192.168.1.254`, MAC `cc:ed:21:66:c7:a0` (OUI da Nokia Shanghai Bell; o Wireshark mostra `NokiaShangha_66:c7:a0`)

Uma particularidade deste PC: a placa física está ligada a um switch virtual do Hyper-V, e o IP fica no adaptador virtual "vEthernet (Hyper-V) 2", que usa o mesmo MAC da placa. A captura foi feita na interface física ("Ethernet"), e não no adaptador virtual, para registrar os quadros como eles passam pelo cabo.

Os nomes `GigaByteTech_…` e `NokiaShangha_…` não estão dentro do quadro: o Wireshark troca os três primeiros bytes do MAC (o OUI, identificador do fabricante atribuído pelo IEEE) pelo nome do fabricante. O endereço completo aparece entre parênteses no painel de detalhes.

Como a captura foi realizada

1. Captura iniciada no Wireshark na interface **Ethernet**, sem filtro de captura.
2. Durante a captura, o tráfego foi gerado no PowerShell (Figura 1):
   - `arp -d *` e `ping 192.168.1.254 -n 4`: ICMP com 32 bytes de dados;
   - `ping 192.168.1.254 -f -l 1472 -n 2`: bit DF ligado, pacote IP de 1500 bytes (20 IP + 8 ICMP + 1472);
   - `ping 192.168.1.254 -f -l 1473 -n 1`: um byte acima do limite. O Windows recusou ("O pacote precisa ser fragmentado, mas a desfragmentação está ativa");
   - navegação web, que gerou o tráfego IPv4/IPv6 comum;
   - `python injetar_quadros.py`: script próprio (Apêndice A) que envia pela mesma placa seis quadros montados byte a byte. Ele foi usado para obter os formatos que não aparecem sozinhos nesta rede: o IEEE 802.3 e o quadro com tag 802.1Q (seções 2 e 4).
3. A captura foi salva e analisada no Wireshark e com o script `analisar_captura.py` (Apêndice C). O script lê os bytes crus de cada quadro e aplica as regras do cabeçalho MAC. O resumo da saída está no Apêndice D.

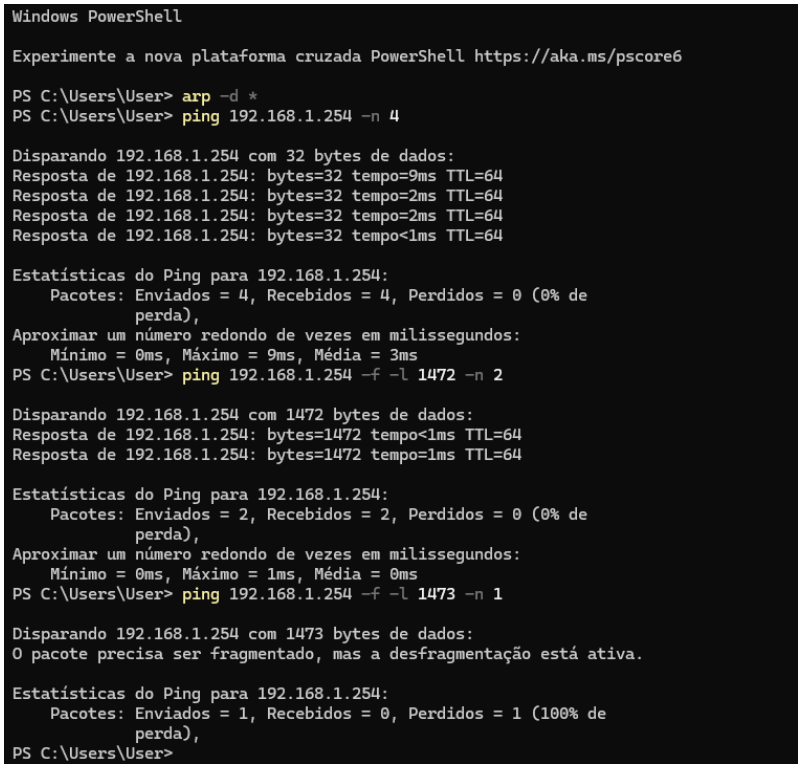


Figura 1 – Execução dos comandos `arp -d *` e `ping` no PowerShell.

Quadros selecionados

Quadro	Sentido	MAC de origem	MAC de destino	Type/Length	Tamanho capturado	Conteúdo	Evidência
13088	enviado	`e0:d5:5e:f4:8e:fa`	`ff:ff:ff:ff:ff:ff` (broadcast)	`0x0806`	42 bytes	ARP request "Who has 192.168.1.254?"	Figura 2
525	recebido	`cc:ed:21:66:c7:a0`	`e0:d5:5e:f4:8e:fa`	`0x0806`	60 bytes	ARP request do roteador "Who has 192.168.1.2?"	Figura 3
13106	enviado	`e0:d5:5e:f4:8e:fa`	`cc:ed:21:66:c7:a0`	`0x0800`	71 bytes	ICMP Echo Request (IP de 57 bytes)	Figura 4
8248	enviado	`e0:d5:5e:f4:8e:fa`	`cc:ed:21:66:c7:a0`	`0x0800`	1514 bytes	ICMP Echo Request do `ping -l 1472` (IP de 1500 bytes, DF)	Figura 5
1	recebido	`cc:ed:21:66:c7:a0`	`e0:d5:5e:f4:8e:fa`	`0x86DD`	102 bytes	IPv6 (segmento TCP)	Apêndice D
4	recebido	`cc:ed:21:66:c7:a0`	`ff:ff:ff:ff:ff:ff` (broadcast)	`0xC0C0`	848 bytes	protocolo não identificado do roteador	Apêndice D

No painel do Wireshark também aparecem os dois bits de controle do primeiro byte de cada endereço:

- **IG bit** (individual/grupo): vale 0 nos endereços unicast e 1 no broadcast `ff:ff:ff:ff:ff:ff` do quadro 13088 ("Group address (multicast/broadcast)").
- **LG bit** (global/local): vale 0 nos MACs de fábrica e 1 no broadcast, onde todos os bits valem 1.

Sobre o quadro 13088: apesar do `arp -d *`, o `ping` não gerou um ARP em broadcast. O roteador envia um ARP request unicast para o PC a cada 5 s (quadros 525, 647, 1063, …). O mais provável é que o pedido de 58,7 s (quadro 7351) tenha chegado entre a limpeza do cache e o primeiro ping (60,9 s). Esse pedido traz o MAC do roteador como remetente, e o Windows o registrou de novo no

cache. O único ARP em broadcast da captura é o 13088, enviado pelo Scapy ao descobrir o MAC do gateway, logo antes da injeção dos quadros de teste.

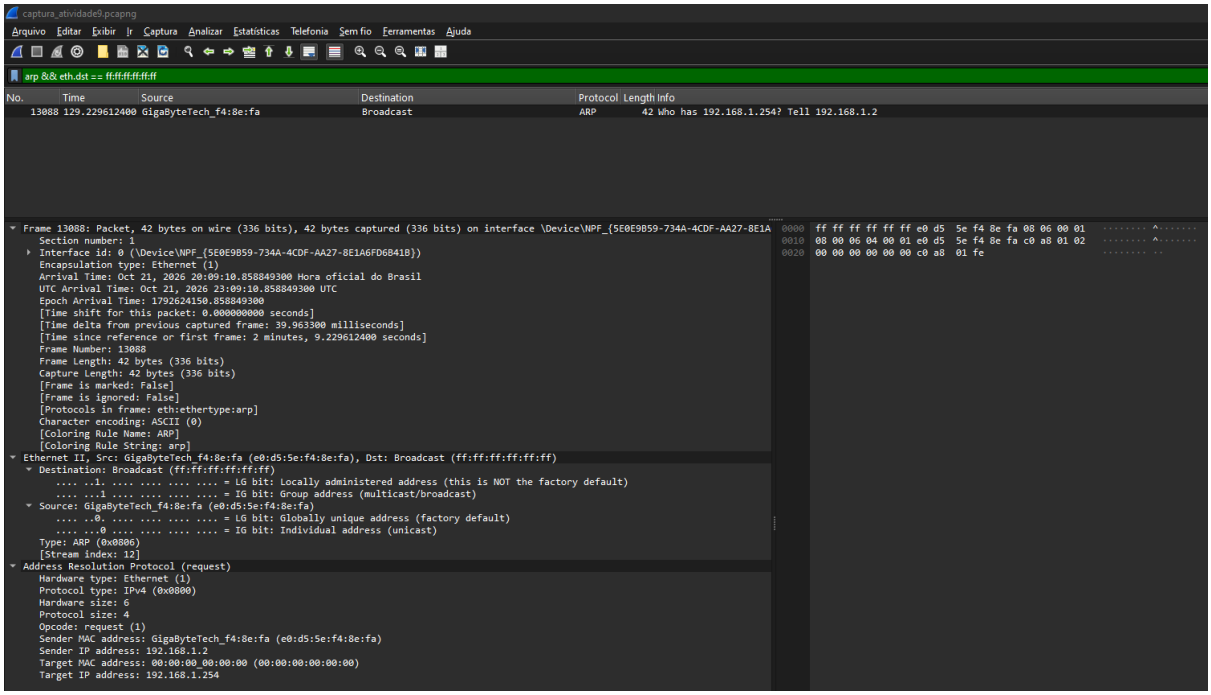


Figura 2 – Quadro 13088 – filtro `arp && eth.dst == ff:ff:ff:ff:ff:ff`, painel Ethernet II expandido.

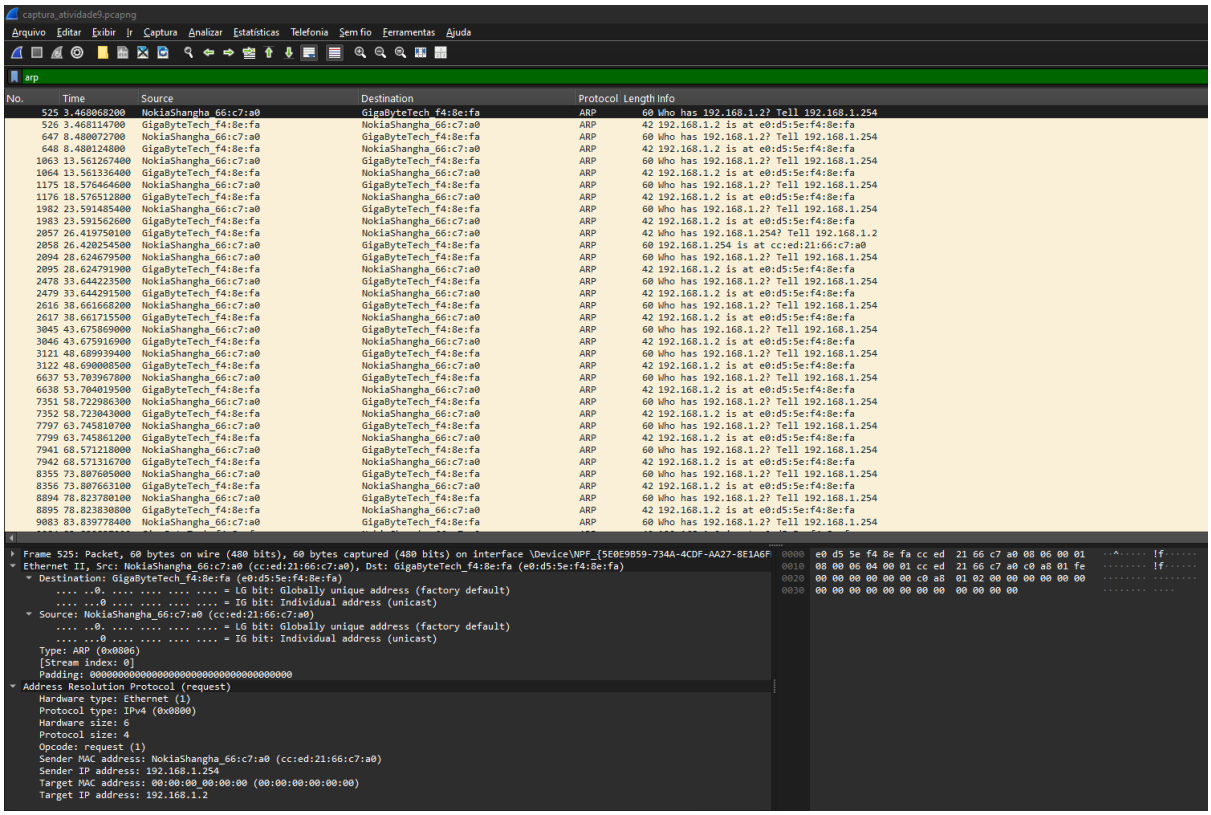


Figura 3 – Quadro 525 – filtro `arp`, painel Ethernet II expandido (com o campo *Padding*).

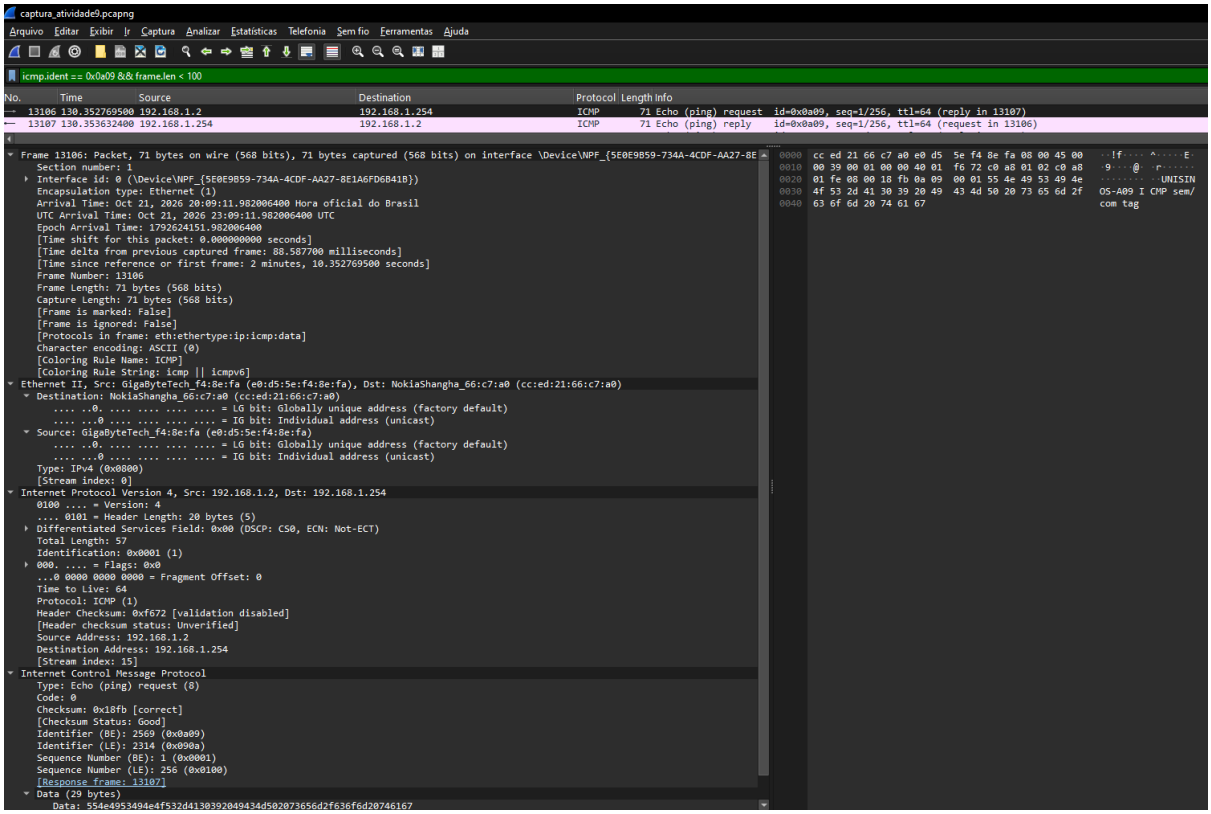


Figura 4 – Quadro 13106 – filtro `icmp.ident == 0x0a09 && frame.len < 100`.

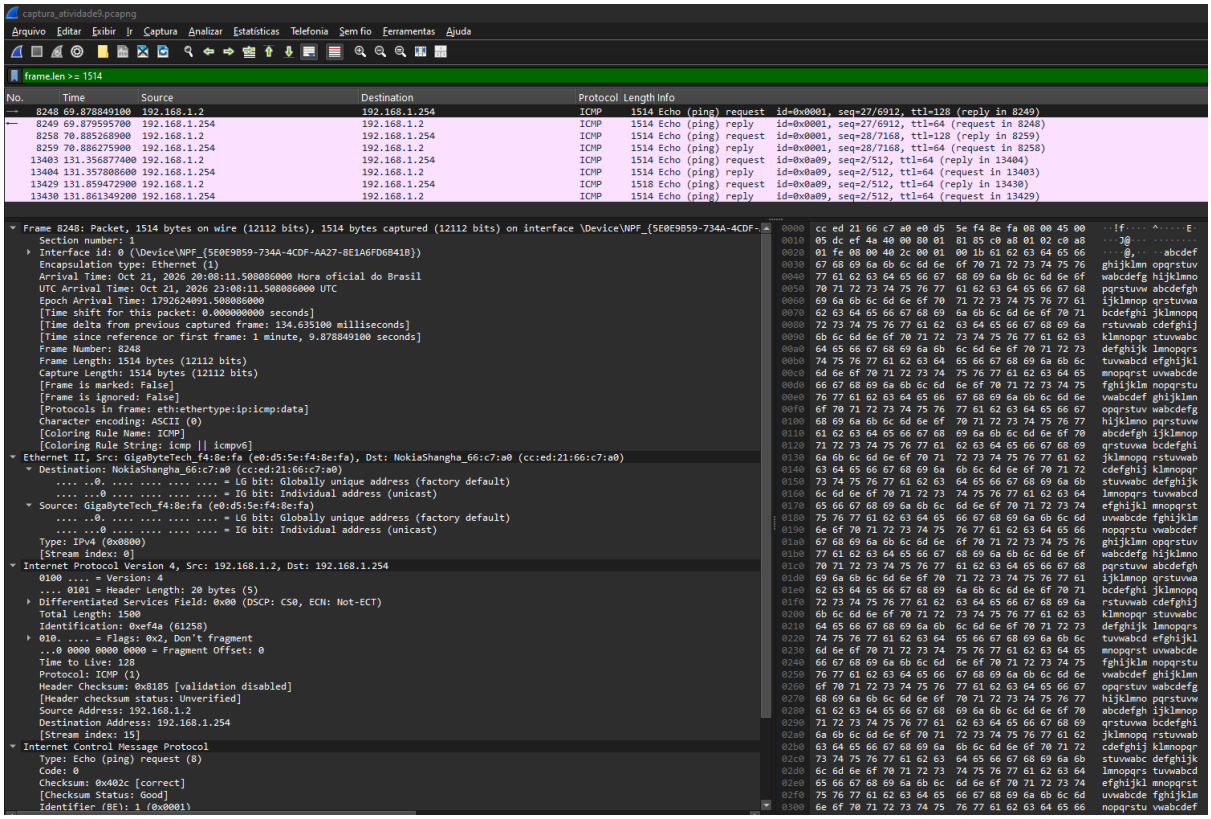


Figura 5 – Quadro 8248 – filtro `frame.len >= 1514`.

2. Formato do quadro MAC

Classificação: Ethernet II (DIX)

Os dois formatos da Aula 11 (slide 33) têm a mesma estrutura: destino (6), origem (6), um campo de 2 bytes, dados (0 a 1500), PAD (0 a 46) e CRC-32 (4). O que muda é o significado do campo de 2 bytes (bytes 12 e 13 do quadro):

- **≤ 1500 (0x05DC):** o campo é **Length**, e o quadro é **IEEE 802.3**;
- **≥ 1536 (0x0600):** o campo é **Type** (EtherType), e o quadro é **Ethernet II (DIX)**;
- de 1501 a 1535: valor não definido em nenhum dos dois formatos.

O limite de 1536 fica acima do maior Length possível (1500), então os dois formatos podem conviver no mesmo cabo sem ambiguidade.

Valores encontrados na captura (saída de `analisar_captura.py`):

Type/Length	Decimal	Classificação	Protocolo	Quadros
`0x86DD`	34 525	Ethernet II	IPv6	11 244
`0x0800`	2 048	Ethernet II	IPv4	2 237
`0xC0C0`	49 344	Ethernet II	não registrado (roteador)	391
`0x0806`	2 054	Ethernet II	ARP	62
`0x88B5`	34 997	Ethernet II	IEEE Local Experimental (quadro de teste)	1
`0x0015`	21	**IEEE 802.3**	LLC (quadro de teste)	1
Total			13 935 Ethernet II + 1 IEEE 802.3	**13 936**

Todo o tráfego gerado pelo sistema operacional e pelo roteador é **Ethernet II**: em todos esses quadros o valor é maior ou igual a 1536. Nos quadros da seção 1, por exemplo, o ARP tem `0x0806` = 2054 e o IPv4 tem `0x0800` = 2048. Esses valores não são tamanhos: o quadro 13106 tem 71 bytes e o campo vale 2048. O Wireshark confirma a classificação ao nomear a camada "Ethernet II" e o campo "Type".

Os 391 quadros `0xC0C0` são enviados pelo roteador para o broadcast (250 quadros) e para o endereço de grupo `01:80:c2:00:10:45` (141 quadros). O valor é um EtherType (≥ 0x0600), mas o Wireshark não o reconhece ("Unknown"), o que indica um protocolo proprietário do fabricante. O destino `01:80:c2:…` pertence ao bloco de endereços do IEEE 802.1 (o Wireshark mostra `IEEE8021Chai_00:10:45`), mas não é um dos endereços de grupo padronizados mais conhecidos, como o `01:80:c2:00:00:00` do Spanning Tree.

Finalidade do campo nos dois formatos

	Ethernet II (DIX)	IEEE 802.3
Nome do campo	Type (EtherType)	Length
O que informa	qual protocolo está no campo de dados (`0x0800` IPv4, `0x0806` ARP, `0x86DD` IPv6…)	quantos bytes de dados úteis existem (cabeçalho LLC + dados), sem contar o PAD
Quem identifica o protocolo de cima	o próprio Type	o cabeçalho **LLC** (IEEE 802.2) no início dos dados: DSAP, SSAP e Controle, 3 bytes (slide 34), ou a extensão SNAP
Como o receptor separa o PAD	pelo tamanho informado pela camada de cima (ex.: Total Length do IPv4)	pelo próprio Length
Uso atual	praticamente todo o tráfego IP	protocolos de controle de enlace, como o Spanning Tree (BPDU, DSAP `0x42`), e protocolos legados

O slide 33 também mostra que o 802.3 original admitia endereços de 2 ou 6 bytes. Na prática só se usam os de 6 bytes, como no quadro 802.3 desta captura.

Como reconhecer o outro formato: quadro IEEE 802.3 gerado

O tráfego natural da rede só tem Ethernet II. Para mostrar como seria o outro formato, o script enviou um quadro IEEE 802.3 com cabeçalho LLC (quadro **13096**, Figura 6):

```
0000  cc ed 21 66 c7 a0  e0 d5 5e f4 8e fa  00 15  00 00 e3  55 4e 49 53 ...
      \__ destino ___/  \__ origem ____/  Len   \LLC /   "UNISINOS-A09 802.3"
```

- Bytes 12 e 13 = `00 15` = **21**, menor que 1500: o campo é **Length**. O Wireshark nomeia a camada "IEEE 802.3 Ethernet" e mostra "Length: 21".
- Os dados começam com 3 bytes de **LLC**: DSAP `0x00` (Null LSAP), SSAP `0x00` e Controle `0xE3` (quadro não numerado, função TEST). O Wireshark mostra a camada "Logical-Link Control".
- Depois do LLC vêm 18 bytes de texto ("UNISINOS-A09 802.3"): 3 + 18 = 21, exatamente o Length.
- A área de dados tem o mínimo de 46 bytes, então os outros 25 bytes são PAD. O Wireshark os separa como "Padding" justamente porque o Length diz onde os dados terminam.
- O roteador não respondeu: não há nenhum outro quadro LLC na captura.

Para comparação, o quadro **13091** (Figura 7) tem a mesma estrutura de cabeçalho, mas com o valor `0x88B5` = 34 997, que é maior que 1536. É um Ethernet II com o EtherType reservado pelo IEEE para experimentos locais. Os 45 bytes de dados vão direto após o Type, sem LLC.

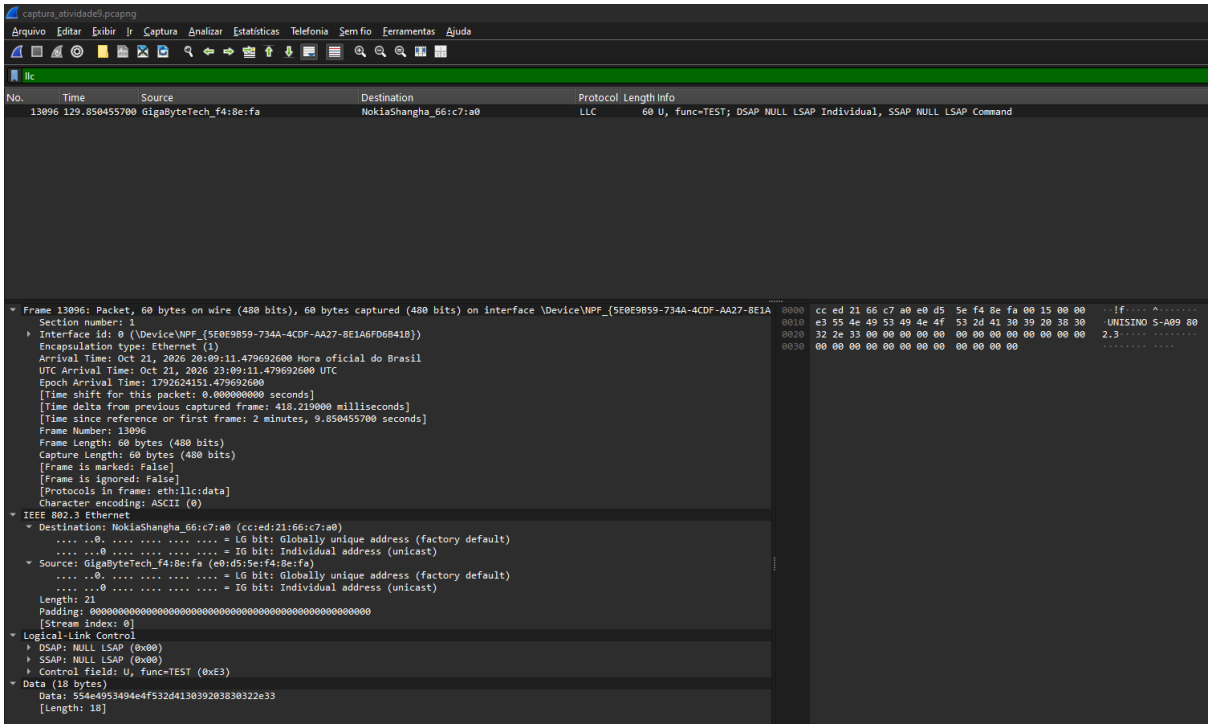


Figura 6 – Quadro 13096 – filtro `llc`, camadas "IEEE 802.3 Ethernet" e "Logical-Link Control" expandidas.

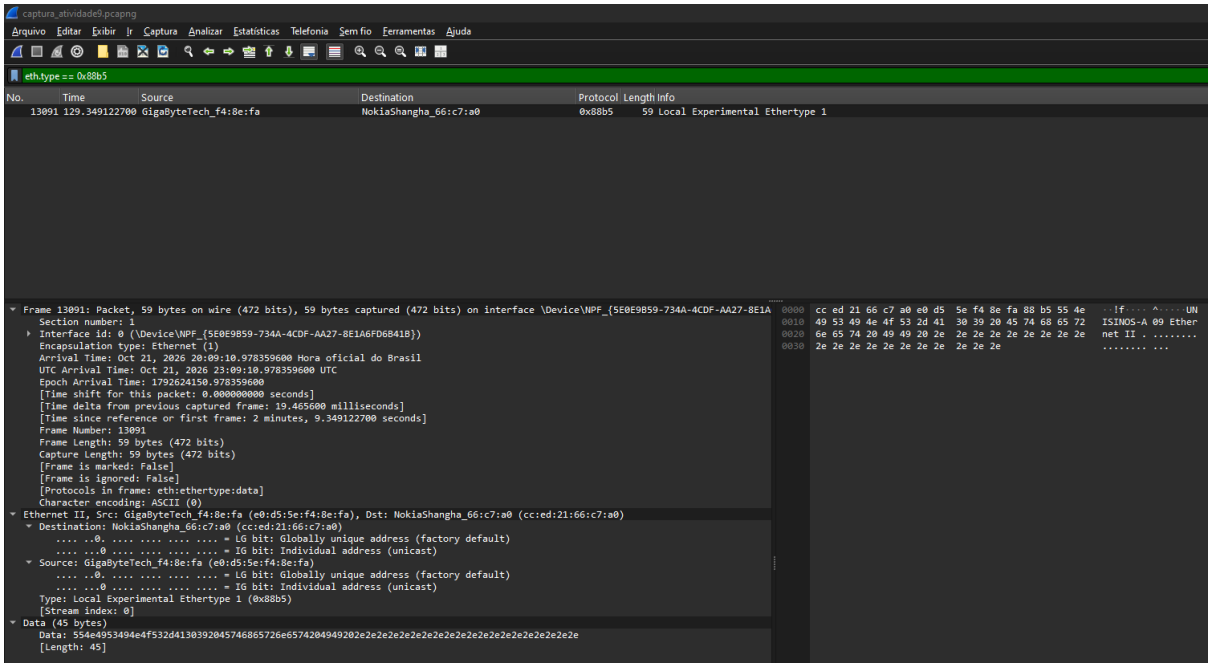


Figura 7 – Quadro 13091 – filtro `eth.type == 0x88b5`.

3. Tamanhos do quadro Ethernet

Estrutura teórica

Campo	Bytes	Pertence ao quadro MAC?	Aparece na captura?
Preâmbulo (7 × 10101010)	7	não (camada física)	não
SFD / SD (10101011)	1	não (camada física)	não
Endereço de destino	6	sim	sim
Endereço de origem	6	sim	sim
Type/Length	2	sim	sim
Dados + PAD	46 a 1500	sim	sim (o PAD só nos quadros recebidos)
FCS (CRC-32)	4	sim	não

- **Cabeçalho Ethernet básico: 14 bytes**: destino (6) + origem (6) + Type/Length (2). Somando o FCS no final do quadro, o overhead do quadro MAC é de 18 bytes.
- **Payload máximo: 1500 bytes** (o MTU da Ethernet, sem jumbo frames). O mínimo é de 46 bytes: dados menores são completados com PAD (0 a 46 bytes, slide 33).
- **Quadro MAC:** de 64 a 1518 bytes, contando do endereço de destino até o FCS.
- **Preâmbulo/SFD:** são 8 bytes a mais, que não fazem parte do quadro MAC. O preâmbulo serve para o receptor sincronizar o relógio com o sinal. O SFD termina em `11` e marca onde começa o endereço de destino. No meio físico, cada quadro ocupa então de **72 a 1526 bytes**, e entre dois quadros ainda há um intervalo obrigatório de 12 bytes (IFG, 96 bits), que não faz parte de nenhum quadro.

Por que alguns campos não aparecem na captura

O Npcap captura os quadros dentro do sistema operacional, entre a pilha de protocolos e o driver da placa:

- **Recepção:** a placa consome o preâmbulo e o SFD (camada física), confere o FCS, descarta o quadro se o CRC estiver errado e remove o FCS antes de entregar o quadro ao sistema. Por isso

nenhum quadro da captura tem o campo FCS, e o maior quadro recebido tem 1514 bytes, e não 1518.

- **Envio:** o quadro é capturado antes de chegar à placa. O **PAD**, o **FCS**, o preâmbulo e o SFD só são acrescentados depois, pelo hardware. Por isso os quadros enviados pelo PC podem aparecer com menos de 60 bytes.

O "bytes on wire" que o Wireshark mostra é, na verdade, o tamanho entregue pelo driver. O quadro 13088 aparece com "42 bytes on wire", mas no cabo ocupou 8 (preâmbulo/SFD) + 60 (com 18 de PAD) + 4 (FCS) = **72 bytes**.

O que a captura mostrou

Sentido	Quadros	Menor	Maior
Enviados pelo PC	4 254	42	1514 sem tag / 1518 com tag
Recebidos	9 682	**60**	**1514**

- Há **405** quadros com menos de 60 bytes, **todos enviados pelo PC**: 364 TCP ACK de 54 bytes, 31 ARP de 42, 4 IGMPv3 de 54, 5 TCP de 55 e o quadro de teste `0x88B5` de 59.
- Nenhum quadro recebido tem menos de 60 bytes. Há **329** com exatamente 60 bytes e PAD: 296 TCP, 31 ARP e 2 IGMPv3.
- O mesmo tipo de quadro aparece com tamanhos diferentes conforme o sentido:
  - ARP request enviado (quadro 13088): **42 bytes** = 14 + 28 de ARP, sem PAD;
  - ARP request recebido (quadro 525): **60 bytes** = 14 + 28 + **18 de PAD**. O Wireshark mostra o campo "Padding: 000…" (Figuras 2 e 3);
  - com o TCP ACK acontece o mesmo: 54 bytes enviado e 60 recebido (6 de PAD).
- O maior quadro sem tag é o 8248, com **1514 bytes** = 14 + 1500 (IPv4 com Total Length 1500 e DF ligado, Figura 5). O ping de 1473 bytes não gerou nenhum quadro: o filtro `ip.len > 1500 || ip.flags.mf == 1 || ip.frag_offset > 0` retorna 0 quadros. O Windows recusou o pacote antes do envio, porque ele não cabia nos 1500 bytes de payload e o DF proibia a fragmentação.

4. Presença de VLAN (IEEE 802.1Q)

Tráfego normal: tag ausente

Dos 13 936 quadros da captura, só os 2 quadros de teste enviados de propósito com tag têm tag VLAN. Nenhum quadro gerado pelo sistema operacional ou pela rede tem tag. Há dois motivos:

1. A rede é doméstica, sem VLANs configuradas: o PC está ligado a uma porta comum (de acesso) do roteador, cujos quadros não são marcados.
2. O driver da placa Intel está com a opção "Prioridade de pacotes e VLAN" habilitada. Nessa configuração, segundo a documentação do Wireshark sobre captura de VLAN, a placa retira a tag por hardware na recepção e a entrega ao sistema como metadado. Mesmo que chegasse um quadro marcado, ele não apareceria com a tag nesta captura. Esse comportamento não foi testado desabilitando a opção.

Estrutura com a tag: quadros gerados

O script enviou o mesmo ICMP Echo Request duas vezes, sem tag (quadro **13106**, Figura 4) e com tag (quadro **13206**, Figura 8). O pacote IP é idêntico nos dois quadros: Total Length 57, Identification 1 e checksum `0xf672`. Só muda o cabeçalho Ethernet:

```
13106 (sem tag):  cc ed 21 66 c7 a0  e0 d5 5e f4 8e fa  08 00        45 00 00 39 ...
13206 (com tag):  cc ed 21 66 c7 a0  e0 d5 5e f4 8e fa  81 00 a0 0a  08 00  45 00 00 39 ...
                  \__ destino ___/  \__ origem ____/  TPID  TCI   Type
```

A tag 802.1Q (slide 34) é inserida entre o endereço de origem e o Type original:

- **TPID** = `0x8100` (2 bytes): ocupa a posição do Type. Por isso o Wireshark mostra, dentro do Ethernet II, "Type: 802.1Q Virtual LAN (0x8100)". Um receptor que encontra `0x8100` nesse campo sabe que os 2 bytes seguintes são o TCI e que o Type verdadeiro vem depois.
- **TCI** = `0xA00A` (2 bytes) = $1010\ 0000\ 0000\ 1010_2$:

Subcampo	Bits	Valor binário	Valor	Significado
PCP (802.1p)	3	`101`	**5**	prioridade; o Wireshark mostra "Voice, < 10ms latency and jitter"
DEI (antigo CFI)	1	`0`	**0**	o quadro não é candidato a descarte em caso de congestionamento
VLAN ID	12	`0000 0000 1010`	**10**	VLAN à qual o quadro pertence (1 a 4094 utilizáveis; 0 e 4095 reservados)

- Depois da tag vem o **Type original** `0x0800` (IPv4), agora nos bytes 16 e 17.

O Wireshark decodifica os mesmos valores: "802.1Q Virtual LAN, PRI: 5, DEI: 0, ID: 10" (Figuras 8 e 9).

Quantos bytes a tag acrescenta

A tag acrescenta **4 bytes** (TPID 2 + TCI 2). Na captura:

- 13106 → 13206: **71 → 75 bytes**, com o mesmo pacote IP;
- 13403 → 13429: **1514 → 1518 bytes**, os dois com payload de 1500 (Figura 9). Com o FCS, o quadro máximo passa de 1518 para **1522 bytes**, valor que o padrão admite para quadros com tag.

O payload máximo continua sendo de 1500 bytes. Como o quadro mínimo continua sendo de 64 bytes, com a tag basta um campo de dados de 42 bytes para atingi-lo.

Comparação dos cabeçalhos

Bytes	Sem tag (13106)	Com tag (13206)
0–5	Destino `cc:ed:21:66:c7:a0`	Destino `cc:ed:21:66:c7:a0`
6–11	Origem `e0:d5:5e:f4:8e:fa`	Origem `e0:d5:5e:f4:8e:fa`
12–13	**Type** `0x0800`	**TPID** `0x8100`
14–15	início do IPv4 (`45 00`)	**TCI** `0xA00A` (PCP 5, DEI 0, VID 10)
16–17	(IPv4)	**Type** `0x0800`
Tamanho do cabeçalho	**14 bytes**	**18 bytes**
Quadro capturado	71 bytes	75 bytes
Máximo sem FCS / com FCS	1514 / 1518	1518 / 1522

Resposta do roteador aos quadros com tag

Esperava-se que o roteador descartasse os quadros da VLAN 10, que não existe na rede. Ele respondeu aos dois (quadros 13207 e 13430), e as respostas vieram sem tag. Elas são iguais às respostas aos pings sem tag: mesmos tamanhos (71 e 1514 bytes) e o mesmo campo DS do IP (`0xFC`) que o roteador coloca em todas as respostas ICMP. Isso sugere que a porta LAN do roteador não filtra quadros pelo VLAN ID. Mas, a partir deste ponto de captura, não é possível afirmar se a tag chegou ao roteador ou se ele respondeu com tag, porque a placa do PC retiraria uma tag na recepção.

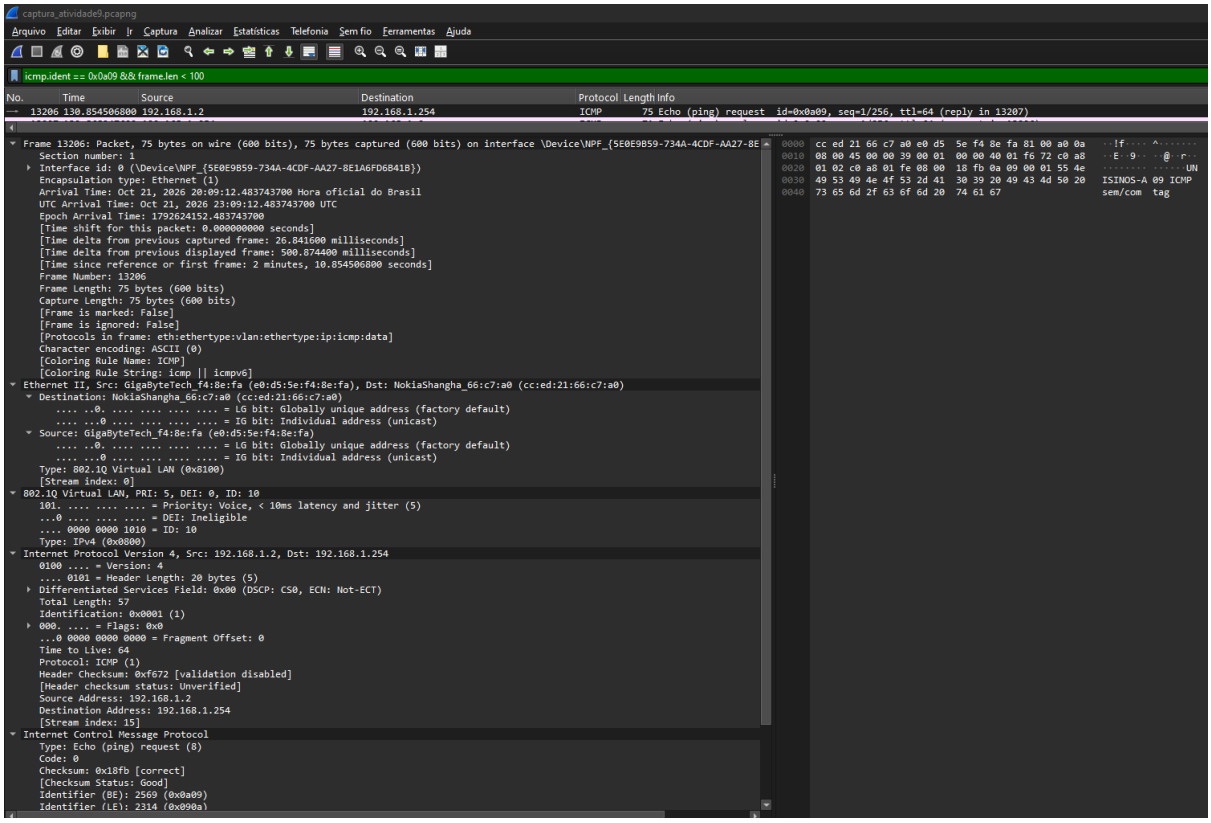


Figura 8 – Quadro 13206 – filtro `icmp.ident == 0x0a09 && frame.len < 100`, camada 802.1Q expandida.

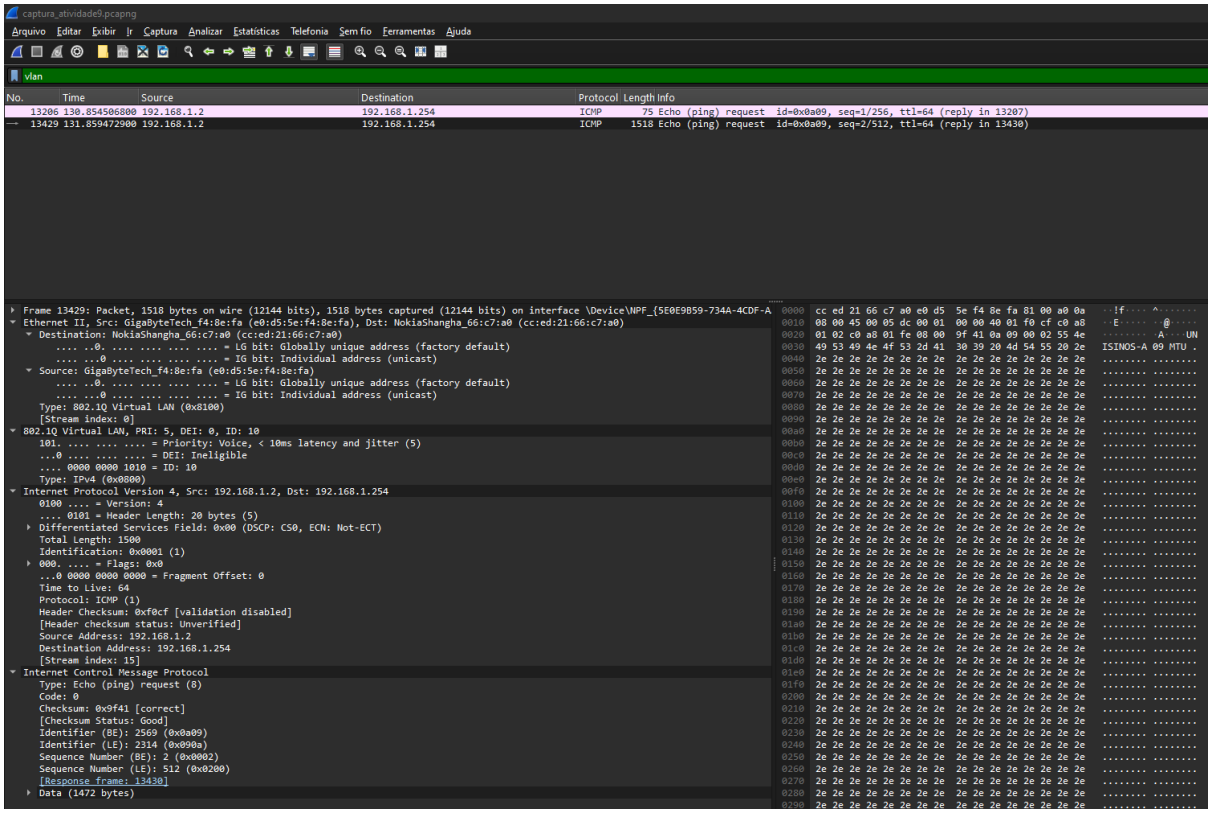


Figura 9 – Quadro 13429 – filtro `vlan`, quadro máximo com tag (1518 bytes).

5. Análise dos resultados

Que evidências permitem diferenciar Ethernet II de IEEE 802.3?

1. **O valor dos bytes 12 e 13**, ou dos bytes 16 e 17 se houver tag: valor ≥ `0x0600` é Type (Ethernet II) e ≤ `0x05DC` é Length (802.3). Na captura, `0x0800`, `0x0806`, `0x86DD`, `0xC0C0` e `0x88B5` caracterizam quadros Ethernet II, e `0x0015` caracteriza o único quadro 802.3.
2. **A coerência do valor com o tamanho do quadro:** no 802.3, o Length (21) coincide com o LLC + dados (3 + 18), e o que sobra (25 bytes) é PAD. No Ethernet II, o valor não tem relação com o tamanho: `0x0800` = 2048 num quadro de 71 bytes.
3. **O início do campo de dados:** no 802.3 vem o cabeçalho LLC (DSAP, SSAP e Controle: `00 00 e3`). No Ethernet II vem diretamente o protocolo indicado pelo Type: `45 …` é o início de um IPv4 e `00 01 08 00 …` é o início de um ARP.

Os endereços não servem para diferenciar os formatos, porque os dois usam 6 bytes de destino e 6 de origem. O Wireshark aplica essa mesma regra para decidir entre "Ethernet II" e "IEEE 802.3 Ethernet" + "Logical-Link Control".

Os tamanhos observados correspondem aos limites teóricos?

Sim, desde que se considere o ponto em que a captura é feita:

- **Quadros recebidos: de 60 a 1514 bytes.** São exatamente os limites teóricos de 64 a 1518 menos os 4 bytes do FCS, que a placa remove. Quadros pequenos chegaram com PAD: 329 quadros de 60 bytes, como o ARP 525 com 18 bytes de PAD.
- **Quadros enviados: de 42 a 1518 bytes.** Ficam abaixo de 60 porque o PAD e o FCS são colocados pela placa depois da captura. O ARP 13088 tem 42 bytes na captura e 64 no cabo, sem contar o preâmbulo. Acima de 1514 só aparecem os 1518 bytes do quadro com tag, dentro do limite de 1522 com FCS do 802.1Q.
- **O payload nunca passou de 1500 bytes.** O ping de 1472 bytes gerou quadros de exatamente 1514 bytes, e o de 1473 foi recusado pelo próprio Windows. Não há jumbo frames na rede.

O tamanho real no cabo é o tamanho capturado + PAD (quando o quadro tem menos de 60 bytes) + 4 de FCS + 8 de preâmbulo/SFD.

Qual é a função da tag VLAN e o que sua presença (ou ausência) indica?

A tag 802.1Q permite dividir uma mesma infraestrutura física de switches em várias redes lógicas independentes, cada uma com o seu domínio de broadcast:

- O **VLAN ID** diz a qual VLAN o quadro pertence, para que os switches o encaminhem só para as portas daquela VLAN. Com isso, um único enlace entre switches (*trunk*) pode carregar o tráfego de várias VLANs.
- O **PCP** (802.1p) define uma prioridade de 0 a 7, usada em QoS para favorecer, por exemplo, tráfego de voz.

A tag normalmente existe só nos enlaces entre switches. O switch acrescenta a tag quando o quadro entra por uma porta de acesso e a retira quando o quadro sai por outra porta de acesso, de modo que os computadores costumam enviar e receber quadros sem tag.

Neste caso, a ausência da tag no tráfego normal indica que o PC está numa porta de acesso de uma rede sem VLANs, ou seja, num único domínio de broadcast. Isso é coerente com o PC receber os broadcasts do roteador (`0xC0C0` e ARP). Há uma ressalva: a placa Intel retira tags na recepção, então a ausência na captura do Windows não prova sozinha que nenhum quadro marcado chegou à placa.

Nos quadros em que a tag está presente (os de teste), ela não altera o restante do quadro: os endereços são os mesmos, o Type `0x0800` aparece logo depois da tag e o pacote IP é idêntico. A tag apenas classifica o quadro (VLAN 10, prioridade 5) para os equipamentos do caminho.

6. Uso de Inteligência Artificial

Ferramenta utilizada: Claude (Anthropic), via Claude Code no VS Code.

Etapas em que ajudou:

- leitura do enunciado e da Aula 11 para planejar a atividade;
- uma captura prévia de teste, para conhecer a rede (interfaces, switch virtual do Hyper-V, tipos de tráfego presentes);
- escrita dos scripts em Python com Scapy, um para gerar os quadros 802.3 e 802.1Q e outro para analisar a captura;
- conferência dos números com o tshark;

Como as sugestões foram verificadas: a captura oficial e os prints foram feitos por mim. Cada valor calculado pelos scripts foi comparado com o que o Wireshark mostra no mesmo quadro. Por exemplo:

- o TCI `0xA00A` calculado pelo script corresponde ao "PRI: 5, DEI: 0, ID: 10" do Wireshark;
- o Length 21 e os 25 bytes de PAD do quadro 13096 aparecem iguais no painel "IEEE 802.3 Ethernet";
- os 18 bytes de PAD do ARP 525 aparecem como "Padding" no painel do quadro.

Previsões confirmadas pela captura:

- o tráfego normal seria só Ethernet II, sem tag VLAN;
- quadros enviados apareceriam com menos de 60 bytes e quadros recebidos com no mínimo 60, com PAD;
- os maiores quadros teriam 1514 bytes sem tag e 1518 com tag;
- o ping de 1473 bytes com DF seria recusado.

Previsões e sugestões que precisaram ser corrigidas:

- **ARP após `arp -d *`:** a IA previu que limpar o cache e fazer o ping geraria um ARP request em broadcast do PC. A captura mostrou que não: entre a limpeza e o ping chegou um dos ARP requests que o roteador envia a cada 5 s, e o Windows voltou a registrar o MAC do roteador a partir dele. O único broadcast ARP da captura (13088) veio do Scapy. O texto da seção 1 foi escrito a partir do que a captura mostrou.
- **Quadros com tag:** a IA previu que o roteador ignoraria os quadros da VLAN 10, mas ele respondeu. Também foi testada a hipótese de que o campo DS das respostas revelaria se a tag tinha chegado. Todas as respostas tinham o mesmo valor `0xFC`, então a questão ficou registrada como inconclusiva (seção 4).
- **Erro no script de análise:** na primeira versão, o script considerava como MAC do PC o MAC de origem mais frequente da captura, e escolheu o do roteador, que num download envia mais quadros que o PC. O erro apareceu porque o resultado dizia que nenhum quadro menor que 60 bytes tinha sido enviado pelo PC, o que contradizia o Wireshark: os ARP de 42 bytes têm origem `GigaByteTech_f4:8e:fa`. O script foi corrigido para usar os MACs das placas do próprio computador.

7. Conclusão

A captura na placa física mostrou que todo o tráfego da rede usa o formato Ethernet II: em todos os quadros do sistema e do roteador, o campo de 2 bytes após os endereços tem valor ≥ `0x0600` e identifica o protocolo de cima (IPv6, IPv4, ARP e um protocolo próprio do roteador). O formato IEEE 802.3 só apareceu no quadro gerado para o teste, e lá a diferença ficou visível: o mesmo campo vale 21, que é o tamanho do LLC + dados, e é ele que permite separar os dados dos 25 bytes de PAD.

Os tamanhos observados batem com os limites teóricos quando se considera o ponto de captura. O cabeçalho tem 14 bytes e o payload no máximo 1500. Preâmbulo, SFD e FCS nunca aparecem, porque são tratados pela placa. E o PAD só aparece nos quadros recebidos, o que faz os quadros enviados parecerem menores que o mínimo de 64 bytes. A tag 802.1Q não aparece no tráfego normal, como esperado numa rede doméstica sem VLANs. No quadro de teste, ela se mostrou como 4 bytes inseridos antes do Type (TPID `0x8100` e TCI com prioridade 5 e VLAN 10), sem alterar o restante do quadro.

Apêndice A – Script de geração dos quadros (`injetar_quadros.py`)

```
"""Gera e envia quadros de teste para a Atividade 09 (Ethernet e MAC).

A captura normal do PC só mostra Ethernet II sem tag VLAN. Este script envia,
pela placa física, quadros montados byte a byte para que o Wireshark capture
também os formatos que não aparecem sozinhos:

  1. ethernet2    Ethernet II com EtherType experimental 0x88B5
  2. ieee8023     IEEE 802.3 + LLC (TEST), com Length < área de dados (PAD)
  3. icmp         ICMP Echo para o gateway, SEM tag (referência)
  4. icmp-vlan    o mesmo ICMP Echo, COM tag 802.1Q (VID 10, PCP 5)
  5. max          Ethernet II com payload de 1500 bytes (maior quadro sem tag)
  6. max-vlan     o mesmo, com tag 802.1Q (maior quadro com tag)

Uso (PowerShell como administrador, Wireshark capturando na "Ethernet"):
  python injetar_quadros.py             envia os 6 quadros
  python injetar_quadros.py --dry-run    só monta e grava quadros_gerados.pcap
  python injetar_quadros.py --so icmp icmp-vlan
"""

import argparse
import sys
import time
from pathlib import Path

from scapy.all import ICMP, IP, LLC, Dot1Q, Dot3, Ether, Raw, conf, sendp, wrpcap

import comum

PASTA = Path(__file__).resolve().parent
MARCA = b"UNISINOS-A09"


def montar(rede, vid, pcp):
    dst, src = rede["gw_mac"], rede["host_mac"]
    ip = IP(src=rede["host_ip"], dst=rede["gw_ip"])
    dados_llc = MARCA + b" 802.3"
    quadros = {}

    quadros["ethernet2"] = (
        "Ethernet II, EtherType 0x88B5 (IEEE Local Experimental)",
        Ether(dst=dst, src=src, type=0x88B5) / Raw(MARCA + b" Ethernet II " + b"." * 20),
    )

    # Length = só LLC + dados; o PAD que completa os 46 bytes não entra na conta
    quadros["ieee8023"] = (
        "IEEE 802.3 + LLC TEST (DSAP/SSAP 0x00), Length menor que a área de dados",
        Dot3(dst=dst, src=src, len=3 + len(dados_llc))
        / LLC(dsap=0x00, ssap=0x00, ctrl=0xE3)
        / Raw(dados_llc)
        / Raw(b"\x00" * (46 - 3 - len(dados_llc))),
    )

    icmp = ICMP(type=8, id=0x0A09, seq=1) / Raw(MARCA + b" ICMP sem/com tag")
    quadros["icmp"] = (
        "ICMP Echo Request para o gateway, sem tag (referência)",
        Ether(dst=dst, src=src) / ip / icmp,
    )
    quadros["icmp-vlan"] = (
        f"ICMP Echo Request idêntico, com tag 802.1Q (VID {vid}, PCP {pcp})",
        Ether(dst=dst, src=src) / Dot1Q(prio=pcp, dei=0, vlan=vid) / ip / icmp,
    )

    # IP Total Length = 1500 (MTU): 20 IP + 8 ICMP + 1472 de dados
    grande = ICMP(type=8, id=0x0A09, seq=2) / Raw((MARCA + b" MTU " + b"." * 1472)[:1472])
    quadros["max"] = (
        "Ethernet II com payload de 1500 bytes (maior quadro sem tag)",
        Ether(dst=dst, src=src) / ip / grande,
    )
    quadros["max-vlan"] = (
        f"Mesmo quadro máximo, com tag 802.1Q (VID {vid}, PCP {pcp})",
        Ether(dst=dst, src=src) / Dot1Q(prio=pcp, dei=0, vlan=vid) / ip / grande,
    )
    return quadros


def descrever(nome, titulo, pkt):
    raw = bytes(pkt)
```

```
    q = comum.decodificar_quadro(raw)
    print(f"[{nome}] {titulo}")
    print(f"  tamanho entregue ao driver: {len(raw)} bytes (sem FCS)")
    for v in q["vlans"]:
        print(f"  tag: TPID 0x{v['tpid']:04x}  TCI 0x{v['tci']:04x}"
              f" -> PCP {v['pcp']}, DEI {v['dei']}, VID {v['vid']}")
    print(f"  Type/Length: 0x{q['type_length']:04x} ({q['type_length']}) -> {q['formato']}")
    print(f"  cabeçalho MAC: {q['cabecalho']} bytes | payload: {len(q['payload'])} bytes")
    if q["pad"]:
        print(f"  dados úteis: {q['dados_uteis']} bytes + PAD {q['pad']} bytes")
    print(comum.dump_hex(raw, limite=32))
    print()


def main():
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--iface", default="Ethernet", help='nome da interface (padrão: "Ethernet")')
    ap.add_argument("--vlan", type=int, default=10, help="VLAN ID da tag (padrão: 10)")
    ap.add_argument("--pcp", type=int, default=5, help="prioridade 802.1p da tag (padrão: 5)")
    ap.add_argument("--so", nargs="+", metavar="QUADRO", help="envia só os quadros escolhidos")
    ap.add_argument("--dry-run", action="store_true", help="não envia, só grava o .pcap")
    args = ap.parse_args()

    if not 0 <= args.vlan <= 4095 or not 0 <= args.pcp <= 7:
        sys.exit("VLAN ID deve estar entre 0 e 4095 e PCP entre 0 e 7.")

    conf.verb = 0
    rede = comum.descobrir_rede(args.iface)
    if not rede["gw_mac"]:
        sys.exit("Não consegui descobrir o MAC do gateway (ARP sem resposta).")

    print(f"Interface : {rede['iface'].name} ({rede['iface'].description})")
    print(f"Host      : {rede['host_mac']}  {rede['host_ip']}")
    print(f"Gateway   : {rede['gw_mac']}  {rede['gw_ip']}")
    print()

    quadros = montar(rede, args.vlan, args.pcp)
    escolhidos = args.so or list(quadros)
    invalidos = [n for n in escolhidos if n not in quadros]
    if invalidos:
        sys.exit(f"Quadro(s) desconhecido(s): {', '.join(invalidos)}. Opções: {', '.join(quadros)}")

    enviados = []
    for nome in escolhidos:
        titulo, pkt = quadros[nome]
        descrever(nome, titulo, pkt)
        enviados.append(pkt)
        if not args.dry_run:
            sendp(pkt, iface=rede["iface"])
            time.sleep(0.5)

    saida = PASTA / "quadros_gerados.pcap"
    wrpcap(str(saida), enviados)
    acao = "montados (dry-run, nada enviado)" if args.dry_run else "enviados"
    print(f"{len(enviados)} quadro(s) {acao}. Cópia em {saida.name}")


if __name__ == "__main__":
    main()
```

Apêndice B – Decodificação do cabeçalho MAC (`comum.py`)

```
"""Funções compartilhadas pelos scripts da Atividade 09 (Ethernet e MAC).

Toda a decodificação do cabeçalho MAC é feita "na mão", byte a byte, a partir
dos bytes crus do quadro, para deixar explícito o que cada campo significa
(Aula 11, slides 33 e 34). O Scapy é usado só para descobrir a interface,
ler/gravar arquivos de captura e enviar quadros.
"""

# Limites do campo Type/Length (IEEE 802.3, cláusula 3.2.6)
MAX_LENGTH = 1500          # 0x05DC: até aqui o campo é Length (IEEE 802.3)
MIN_ETHERTYPE = 1536       # 0x0600: a partir daqui o campo é Type (Ethernet II / DIX)

TPID_8021Q = 0x8100        # Tag Protocol Identifier da IEEE 802.1Q
TPID_8021AD = 0x88A8       # Q-in-Q (IEEE 802.1ad), só para reconhecer

# Tamanhos do quadro (bytes)
PREAMBULO = 7
SFD = 1
CABECALHO = 14             # destino 6 + origem 6 + Type/Length 2
TAG_VLAN = 4               # TPID 2 + TCI 2
FCS = 4
PAYLOAD_MIN = 46
PAYLOAD_MAX = 1500
QUADRO_MIN = 64            # com FCS
QUADRO_MAX = 1518          # com FCS, sem tag
QUADRO_MAX_VLAN = 1522     # com FCS, com tag 802.1Q

ETHERTYPES = {
    0x0800: "IPv4",
    0x0806: "ARP",
    0x86DD: "IPv6",
    0x8100: "IEEE 802.1Q (VLAN)",
    0x88A8: "IEEE 802.1ad (Q-in-Q)",
    0x88CC: "LLDP",
    0x888E: "EAPOL (802.1X)",
    0x8863: "PPPoE Discovery",
    0x8864: "PPPoE Session",
    0x893A: "IEEE 1905.1",
    0x88B5: "IEEE Local Experimental 1",
    0x88B6: "IEEE Local Experimental 2",
    0x8899: "Realtek RRCP",
    0x88E1: "HomePlug AV",
}

SAPS_LLC = {
    0x00: "Null SAP",
    0x42: "Spanning Tree (BPDU)",
    0xAA: "SNAP",
    0xE0: "Novell IPX",
    0xF0: "NetBIOS",
    0xFE: "ISO Network Layer",
}


def nome_ethertype(valor):
    return ETHERTYPES.get(valor, "desconhecido/proprietário")


def mac_str(b):
    return ":".join(f"{x:02x}" for x in b)


def tipo_endereco(mac):
    """Classifica um MAC pelos bits I/G e U/L do primeiro octeto."""
    if mac == b"\xff" * 6:
        return "broadcast"
    grupo = "multicast" if mac[0] & 0x01 else "unicast"
    admin = "local" if mac[0] & 0x02 else "global (OUI)"
    return f"{grupo}, administrado {admin}"


def classificar_type_length(valor):
    """Regra que diferencia Ethernet II de IEEE 802.3 pelo valor do campo."""
    if valor <= MAX_LENGTH:
        return "IEEE 802.3", f"{valor} <= 1500 (0x05DC): o campo é Length"
    if valor >= MIN_ETHERTYPE:
        return "Ethernet II (DIX)", f"{valor} >= 1536 (0x0600): o campo é EtherType"
```

```
    return "indefinido", f"{valor} está entre 1501 e 1535: valor inválido nos dois formatos"


def decodificar_tci(tci):
    return {
        "pcp": tci >> 13,            # 802.1p, 3 bits
        "dei": (tci >> 12) & 0x1,    # antigo CFI, 1 bit
        "vid": tci & 0x0FFF,         # VLAN ID, 12 bits
    }


def decodificar_quadro(raw):
    """Separa o cabeçalho MAC de um quadro cru (sem preâmbulo/SFD e sem FCS)."""
    q = {
        "tamanho": len(raw),
        "dst": raw[0:6],
        "src": raw[6:12],
        "vlans": [],
    }
    pos = 12
    valor = int.from_bytes(raw[pos:pos + 2], "big")
    # Pode haver uma ou mais tags (802.1Q / Q-in-Q) antes do Type/Length real
    while valor in (TPID_8021Q, TPID_8021AD) and len(raw) >= pos + 6:
        tci = int.from_bytes(raw[pos + 2:pos + 4], "big")
        q["vlans"].append({"tpid": valor, "tci": tci, "offset": pos, **decodificar_tci(tci)})
        pos += 4
        valor = int.from_bytes(raw[pos:pos + 2], "big")
    q["type_length"] = valor
    q["offset_tl"] = pos
    q["cabecalho"] = pos + 2
    q["formato"], q["justificativa"] = classificar_type_length(valor)
    q["payload"] = raw[pos + 2:]
    q.update(_analisar_payload(q))
    return q


def _analisar_payload(q):
    """Calcula quantos bytes do payload são dados úteis e quantos são PAD."""
    p, tl = q["payload"], q["type_length"]
    info = {"protocolo": None, "dados_uteis": None, "pad": None, "llc": None}
    if q["formato"] == "IEEE 802.3":
        info["dados_uteis"] = tl
        info["pad"] = max(len(p) - tl, 0)
        if p[:2] == b"\xff\xff":
            info["protocolo"] = "Novell raw 802.3 (IPX sem LLC)"
        elif len(p) >= 3:
            dsap, ssap, ctrl = p[0], p[1], p[2]
            info["llc"] = {"dsap": dsap, "ssap": ssap, "controle": ctrl}
            info["protocolo"] = "LLC" + (" + SNAP" if dsap == 0xAA else "")
        return info
    if q["formato"] != "Ethernet II (DIX)":
        return info
    info["protocolo"] = nome_ethertype(tl)
    # O Ethernet II não informa o tamanho dos dados; quem sabe é a camada de cima
    if tl == 0x0800 and len(p) >= 4:
        info["dados_uteis"] = int.from_bytes(p[2:4], "big")      # IPv4 Total Length
    elif tl == 0x86DD and len(p) >= 6:
        info["dados_uteis"] = 40 + int.from_bytes(p[4:6], "big")  # IPv6 header + Payload Length
    elif tl == 0x0806:
        info["dados_uteis"] = 28                                  # ARP sobre Ethernet/IPv4
    if info["dados_uteis"] is not None:
        info["pad"] = max(len(p) - info["dados_uteis"], 0)
    return info


def dump_hex(raw, limite=64):
    linhas = []
    for i in range(0, min(len(raw), limite), 16):
        bloco = raw[i:i + 16]
        hexa = " ".join(f"{x:02x}" for x in bloco)
        linhas.append(f"  {i:04x}  {hexa}")
    if len(raw) > limite:
        linhas.append(f"  ... (+{len(raw) - limite} bytes)")
    return "\n".join(linhas)


def descobrir_rede(iface_nome="Ethernet"):
    """Retorna a interface de captura/injeção, o MAC/IP do host e o gateway.
```

```
    Neste PC a placa física ("Ethernet") está ligada a um switch virtual do
    Hyper-V, então o IP fica na "vEthernet (Hyper-V) 2" (mesmo MAC). Por isso
    o IP vem da rota padrão e a interface de injeção é escolhida pelo nome.
    """
    from scapy.all import conf, getmacbyip

    iface = conf.ifaces.dev_from_name(iface_nome)
    _, host_ip, gw_ip = conf.route.route("8.8.8.8")
    gw_mac = getmacbyip(gw_ip) if gw_ip and gw_ip != "0.0.0.0" else None
    return {
        "iface": iface,
        "host_mac": iface.mac,
        "host_ip": host_ip,
        "gw_ip": gw_ip,
        "gw_mac": gw_mac,
    }
```

Apêndice C – Script de análise da captura (`analisar_captura.py`)

```
"""Analisa uma captura (.pcap/.pcapng) do Wireshark para a Atividade 09.

Para cada quadro, lê os bytes crus e aplica a regra do campo Type/Length
(<= 1500 -> IEEE 802.3, >= 1536 -> Ethernet II), procura tags 802.1Q
(TPID 0x8100) e confere os tamanhos contra os limites teóricos.

Uso:
  python analisar_captura.py captura.pcapng
  python analisar_captura.py captura.pcapng --quadros 12 57 230
  python analisar_captura.py captura.pcapng --saida saida_analise.txt

Sem --quadros, escolhe automaticamente um exemplo de cada categoria
(ARP, IPv4, IPv6, 802.3, VLAN, quadros gerados pelo injetar_quadros.py...).
Os números de quadro são os mesmos da coluna "No." do Wireshark.
"""

import argparse
import io
import sys
from collections import Counter
from pathlib import Path

from scapy.all import PcapNgReader, PcapReader

import comum


def ler_quadros(caminho):
    with open(caminho, "rb") as f:
        magica = f.read(4)
    leitor = PcapNgReader if magica == b"\x0a\x0d\x0d\x0a" else PcapReader
    with leitor(str(caminho)) as r:
        return [bytes(p) for p in r]


def detectar_host(quadros, host_mac):
    if host_mac:
        return bytes.fromhex(host_mac.replace(":", "").replace("-", ""))
    # Origem mais frequente entre os MACs das placas deste PC. Sem filtrar,
    # o gateway costuma ganhar (num download ele envia mais quadros que o host).
    from scapy.all import conf

    locais = {bytes.fromhex(i.mac.replace(":", "")) for i in conf.ifaces.values() if i.mac}
    origens = Counter(q[6:12] for q in quadros)
    for mac, _ in origens.most_common():
        if mac in locais:
            return mac
    return origens.most_common(1)[0][0]


def categoria(q):
    if q["vlans"]:
        return "VLAN 802.1Q"
    if q["formato"] == "IEEE 802.3":
        return "IEEE 802.3"
    if q["formato"] != "Ethernet II (DIX)":
        return "indefinido"
    if q["type_length"] == 0x88B5:
        return "EtherType 0x88B5 (gerado)"
    if q["type_length"] == 0x0800 and q["tamanho"] >= 1514:
        return "IPv4 máximo"
    return q["protocolo"]


def resumo(quadros, decod, host, out):
    p = lambda *a: print(*a, file=out)
    enviados = [q for q in decod if q["src"] == host]
    recebidos = [q for q in decod if q["src"] != host]

    p("=" * 72)
    p("RESUMO DA CAPTURA")
    p("=" * 72)
    p(f"Quadros: {len(decod)}  (enviados pelo host: {len(enviados)}, recebidos: {len(recebidos)})")
    p(f"MAC do host: {comum.mac_str(host)}")
    p()

    p("Formato do quadro MAC (regra do campo Type/Length):")
```

```
    for fmt, n in Counter(q["formato"] for q in decod).most_common():
        p(f"  {fmt:<22} {n:>7}")
    p()

    p("Valores do campo Type/Length (após eventuais tags):")
    p(f"  {'valor':<8} {'decimal':>7}  {'formato':<18} {'protocolo':<28} {'qtd':>6}")
    tipos = Counter((q["type_length"], q["formato"], q["protocolo"] or "-") for q in decod)
    for (tl, fmt, proto), n in tipos.most_common():
        p(f"  0x{tl:04x}   {tl:>7}  {fmt:<18} {proto:<28} {n:>6}")
    p()

    tagged = [q for q in decod if q["vlans"]]
    p(f"Quadros com tag VLAN (TPID 0x8100/0x88A8): {len(tagged)}")
    for vid, n in Counter(v["vid"] for q in tagged for v in q["vlans"]).most_common():
        p(f"  VLAN ID {vid}: {n} quadro(s)")
    p()

    p("Tamanhos capturados (bytes, sem preâmbulo/SFD e sem FCS):")
    for rotulo, grupo in (("todos", decod), ("enviados", enviados), ("recebidos", recebidos)):
        if grupo:
            t = [q["tamanho"] for q in grupo]
            p(f"  {rotulo:<10} min {min(t):>5}  max {max(t):>5}  média {sum(t) / len(t):8.1f}")
    abaixo = [q for q in decod if q["tamanho"] < 60]
    acima = [q for q in decod if q["tamanho"] > 1514 + comum.TAG_VLAN * len(q["vlans"])]
    p(f"  < 60 bytes (abaixo do mínimo sem FCS): {len(abaixo)}"
      f"  -> enviados pelo host: {sum(q['src'] == host for q in abaixo)}")
    p(f"  > 1514 bytes (+4 por tag): {len(acima)}")
    com_pad = [q for q in recebidos if q["pad"]]
    p(f"  recebidos com PAD: {len(com_pad)}"
      + (f" (ex.: {com_pad[0]['tamanho']} bytes, {com_pad[0]['pad']} de PAD)" if com_pad else ""))
    p()


def detalhar(n, q, host, out):
    p = lambda *a: print(*a, file=out)
    sentido = "enviado pelo host" if q["src"] == host else "recebido"
    p("-" * 72)
    p(f"Quadro nº {n}  |  {q['tamanho']} bytes capturados  |  {sentido}")
    p("-" * 72)
    p(f"  Destino : {comum.mac_str(q['dst'])}  ({comum.tipo_endereco(q['dst'])})")
    p(f"  Origem  : {comum.mac_str(q['src'])}  ({comum.tipo_endereco(q['src'])})")
    for v in q["vlans"]:
        p(f"  Tag     : TPID 0x{v['tpid']:04x}  TCI 0x{v['tci']:04x}"
          f"  -> PCP {v['pcp']} | DEI {v['dei']} | VLAN ID {v['vid']}"
          f"  (bytes {v['offset']}-{v['offset'] + 3})")
    tl = q["type_length"]
    p(f"  Type/Len: 0x{tl:04x} = {tl}  (bytes {q['offset_tl']}-{q['offset_tl'] + 1})")
    p(f"  Formato : {q['formato']}  [{q['justificativa']}]")
    if q["formato"] == "IEEE 802.3" and q["llc"]:
        l = q["llc"]
        p(f"  LLC     : DSAP 0x{l['dsap']:02x} ({comum.SAPS_LLC.get(l['dsap'], '?')})"
          f"  SSAP 0x{l['ssap']:02x}  Controle 0x{l['controle']:02x}")
    elif q["protocolo"]:
        p(f"  Protocolo da camada acima: {q['protocolo']}")
    p(f"  Cabeçalho MAC: {q['cabecalho']} bytes  |  payload: {len(q['payload'])} bytes")
    if q["dados_uteis"] is not None:
        fonte = "campo Length" if q["formato"] == "IEEE 802.3" else "camada superior"
        p(f"  Dados úteis: {q['dados_uteis']} bytes ({fonte})  |  PAD: {q['pad']} bytes")
    no_fio = max(q["tamanho"], 60) + comum.FCS
    p(f"  No meio físico: {comum.PREAMBULO + comum.SFD} (preâmbulo+SFD) + {no_fio} (quadro MAC com"
      f" FCS{' e PAD da placa' if q['tamanho'] < 60 else ''}) = {comum.PREAMBULO + comum.SFD + no_fio}
bytes")
    raw = bytes(q["dst"] + q["src"]) + b"".join(
        v["tpid"].to_bytes(2, "big") + v["tci"].to_bytes(2, "big") for v in q["vlans"]
    ) + tl.to_bytes(2, "big") + q["payload"]
    p("  Bytes iniciais:")
    p(comum.dump_hex(raw, limite=48))
    p()


def escolher_exemplos(decod):
    vistos, escolhidos = set(), []
    for i, q in enumerate(decod, start=1):
        c = categoria(q)
        if c not in vistos:
            vistos.add(c)
            escolhidos.append(i)
    return escolhidos
```

```
def main():
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("captura", type=Path)
    ap.add_argument("--quadros", nargs="+", type=int, metavar="N", help="números dos quadros a detalhar")
    ap.add_argument("--host-mac", help="MAC do host (padrão: origem mais frequente)")
    ap.add_argument("--saida", type=Path, help="grava o relatório em UTF-8 neste arquivo")
    args = ap.parse_args()

    if not args.captura.exists():
        sys.exit(f"Arquivo não encontrado: {args.captura}")
    quadros = ler_quadros(args.captura)
    if not quadros:
        sys.exit("A captura está vazia.")
    decod = [comum.decodificar_quadro(q) for q in quadros]
    host = detectar_host(quadros, args.host_mac)

    out = io.StringIO()
    resumo(quadros, decod, host, out)
    nums = args.quadros or escolher_exemplos(decod)
    print("=" * 72, file=out)
    print("QUADROS DETALHADOS" + ("" if args.quadros else " (um exemplo de cada categoria)"), file=out)
    print("=" * 72, file=out)
    for n in nums:
        if not 1 <= n <= len(decod):
            print(f"Quadro {n} não existe (a captura tem {len(decod)}).\n", file=out)
            continue
        detalhar(n, decod[n - 1], host, out)

    texto = out.getvalue()
    print(texto, end="")
    if args.saida:
        args.saida.write_text(texto, encoding="utf-8")
        print(f"Relatório gravado em {args.saida}")


if __name__ == "__main__":
    main()
```

Apêndice D – Resumo da análise da captura (`saida_analise.txt`)

Saída de `python analisar_captura.py captura_atividade9.pcapng --saida saida_analise.txt` (parte do resumo; o arquivo completo também detalha um quadro de cada categoria, entre eles os quadros 1 e 4 da seção 1).

```
======================================================================
RESUMO DA CAPTURA
======================================================================
Quadros: 13936  (enviados pelo host: 4254, recebidos: 9682)
MAC do host: e0:d5:5e:f4:8e:fa

Formato do quadro MAC (regra do campo Type/Length):
  Ethernet II (DIX)        13935
  IEEE 802.3                   1

Valores do campo Type/Length (após eventuais tags):
  valor    decimal  formato            protocolo                       qtd
  0x86dd     34525  Ethernet II (DIX)  IPv6                          11244
  0x0800      2048  Ethernet II (DIX)  IPv4                           2237
  0xc0c0     49344  Ethernet II (DIX)  desconhecido/proprietário       391
  0x0806      2054  Ethernet II (DIX)  ARP                              62
  0x88b5     34997  Ethernet II (DIX)  IEEE Local Experimental 1         1
  0x0015        21  IEEE 802.3         LLC                              1

Quadros com tag VLAN (TPID 0x8100/0x88A8): 2
  VLAN ID 10: 2 quadro(s)

Tamanhos capturados (bytes, sem preâmbulo/SFD e sem FCS):
  todos      min    42  max  1518  média    810.0
  enviados   min    42  max  1518  média    282.1
  recebidos  min    60  max  1514  média   1042.0
  < 60 bytes (abaixo do mínimo sem FCS): 405  -> enviados pelo host: 405
  > 1514 bytes (+4 por tag): 0
  recebidos com PAD: 329 (ex.: 60 bytes, 6 de PAD)
```